

Chris Roy
Professor McManus
ITAI 1378
09/17/2026

Reflective Journal

L04 Chihuahua or Muffin – A workshop about image classification with a neural network. Here I learned more about how to prepare image data for a neural network. Also the basic structure of a neural network was explained to me again. We created a model with an input layer, several hidden layers and an output layer with 2 classes: chihuahua and muffin. Each layer of the model was equipped with activation functions (in the hidden layers ReLU and in the output layer a softmax function). The model was used with CrossEntropyLoss and an SGD (stochastic gradient descent) optimizer. I learned about these concepts from the PyTorch documentation (PyTorch, 2026).

A few lessons were covered at the start of the workshop about how to import and load images into the neural network. This included how to resize images and turn them into tensors which is what the neural network can understand. The images for the workshop were located in a training folder and a validation folder. The model was tested on the images in the validation folder in order to get an idea of how well the model was doing on images that it had not been trained on.

This lesson taught me about the different layers of a neural network and how to create one. I created a simple neural network with an input layer, a few hidden layers and an output layer with two classes, chihuahua and muffin. Each of the hidden layers in the network used ReLU activation functions. The output layer used a softmax function in order to return a probability for each of the classes in the output layer. I used CrossEntropyLoss for the loss function and stochastic gradient descent (SGD) as the optimizer for the model. CrossEntropyLoss and optimizers are used to change the model parameters in order to get the lowest loss possible for the model (PyTorch, 2026).

The basic structure of the neural network was explained as a typical setup for an image classification problem: input layer followed by a few hidden layers and ending with an output layer with two classes (chihuahua and muffin). Every hidden layer uses ReLU activation functions between the layers. The output layer uses a softmax activation function to compute the probability for every class (i.e. two classes in this case). The model is then trained using the CrossEntropyLoss function and stochastic gradient descent (SGD) as the optimizer. PyTorch documentation states that cross-entropy loss is typically used for classification problems, and optimizers update the model's parameters based on the computed loss (PyTorch, 2026). As I was actually able to watch this code run in the notebook as opposed to just reading about it, I understand the concepts much better now.

As mentioned above, when we tried to run the PyTorch import cell, we encountered a `ModuleNotFoundError` as PyTorch was not installed in our Jupyter virtual environment. We questioned whether the packages we installed using JupyterLab terminal would be installed in our global Python installation. To confirm, we checked our environment and found that the JupyterLab terminal is using our virtual environment `~/python_environments/jupyter`. Thus, we installed PyTorch and Torchvision in this virtual environment and everything worked fine after that. This was also a good lesson to me because I learned that we need to know which Python environment we are using when installing packages.

Once I finished writing all the missing parts of the notebook the training then ran fine. After installation of Jupyter widget support for the required IPywidgets support for the old ``tqdm`` progress-bar functions (which were marked as deprecated by a ``TqdmDeprecationWarning``) in the virtual environment for Jupyter, everything ran without problems. The Python environment and all the required supporting packages for a machine-learning project can cause a lot of problems. Most of the problems one encounters when working with a machine-learning project are not even in the code for the machine learning itself.

After filling in the rest of the missing values and running them, I trained the model to run for three epochs. The results for the training accuracy were 75.00% for the first, 84.17% for the second, and 89.17% for the third. For the validation set accuracy, I saw an increase from 63.33% to 70.00%, and then 83.33% at the end. The final validation run of the model correctly classified 25 out of the 30 images in the validation set. Understanding the difference between the accuracy of training data and the validation data is very important for creating effective models, because it is possible for a model to have a high accuracy for the training data but poor performance for new data.

When displaying the predictions on the validation set for this chihuahua or muffin classifier, the Chihuahua or Muffin workshop reported that all muffin images had been correctly classified. However, some of the chihuahuas were misclassified as muffins. Even in cases where the model made errors, it was extremely confident in its choice for both correctly classified and misclassified images. This confidence in prediction mistakes gave me valuable insight. Specifically, I can rely on a model having high confidence in its choice for images in the test set, however this does not necessarily mean that the choice made by the model is correct.

Using a trained model, it is also possible to improve it using for example the number of epochs, learning rate, optimizer, number of layers etc. as well as changes to the input images such as scaling them down or changing them in some way. Again, simply getting a model to fit the training data is not enough and it is also important to evaluate its performance on unseen data. Common measures for this are accuracy, precision, recall and F1 score.

Overall this was a fantastic workshop with a wide variety of potential applications for Image Classification techniques including organizing a large collection of photos, object recognition on a manufacturing line, monitoring agricultural growth, sorting products on a production line, or even analyzing medical images such as MRIs and CT scans. However, this simple workshop only showed us a single example of how to use Image Classification: to recognize chihuahuas or muffins!

The workshop helped me much better understand the workings of a neural network that has been trained for image classification in detail. The picture was much clearer than before the notebook had been worked through in that respect. In addition to understanding the machine learning itself, I also gained a better understanding of what has to be done to prepare the data, to manage the Python environment and the various supporting packages, to choose a loss function and an optimizer, and how to check the results of a model that has been trained. The various problems that I had, in particular the few that I had with Jupyter, have also helped me to much better work with Python environments and with Jupyter Notebooks in particular.

References

“3.4. Metrics and Scoring: Quantifying the Quality of Predictions.” *Scikit*, scikit-learn.org/stable/modules/model_evaluation.html. Accessed 17 Sept. 2026.

“Intro.” *Training a Classifier - PyTorch Tutorials 2.14.0+cu130 Documentation*, 20 July 2022, docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html.

“Reference Api.” *CrossEntropyLoss - PyTorch 2.14 Documentation*, docs.pytorch.org/docs/2.14/generated/torch.nn.CrossEntropyLoss.html. Accessed 17 Sept. 2026.

“Reference Api.” *Torch.Utills.Data - PyTorch Main Documentation*, docs.pytorch.org/docs/main/data.html. Accessed 17 Sept. 2026.